INSTITUTO FEDERAL DO RIO GRANDE DO NORTE — CAMPUS NATAL ZONA LESTE

Projeto Integrador: Patinhas (Etapa 3)

Arquitetura Distribuída, API HTTP/REST, Persistência Relacional e OpenAPI 3.0

Discente: Lucas Aprígio da Silva | **Matrícula:** 20232158060024

Repositório Oficial (GitHub): https://github.com/lucasaprigiodev/projeto-integrador-patinhas

Documentação Interativa OpenAPI (Swagger UI): https://lucasaprigiodev.github.io/projeto-integrador-patinhas/swagger.html (ou `http://localhost:3000/api-docs`)

Arquivos Principais no Repositório: `backend/server.js` (API REST), `backend/database.js` + `backend/schema.sql` + `backend/patinhas.sqlite` (Banco Relacional), `backend/openapi.json` + `swagger.html` (OpenAPI 3.0), `www/db.js` (Cliente HTTP).

1. Diagrama da Arquitetura Distribuída

O sistema **Patinhas** foi evoluído para uma arquitetura cliente-servidor distribuída em três camadas desacopladas. As interfaces de usuário (Aplicativo Mobile Android empacotado com Capacitor e Painel Web Administrativo da ONG) não manipulam dados isoladamente: todas as operações passam pelo módulo de comunicação HTTP (`www/db.js`), que envia requisições padronizadas em JSON com autenticação JWT para o servidor **API HTTP/REST** (`backend/server.js`), responsável por validar regras de negócio, controlar permissões (RBAC) e persistir os registros no banco de dados relacional **SQLite** (`backend/patinhas.sqlite`).

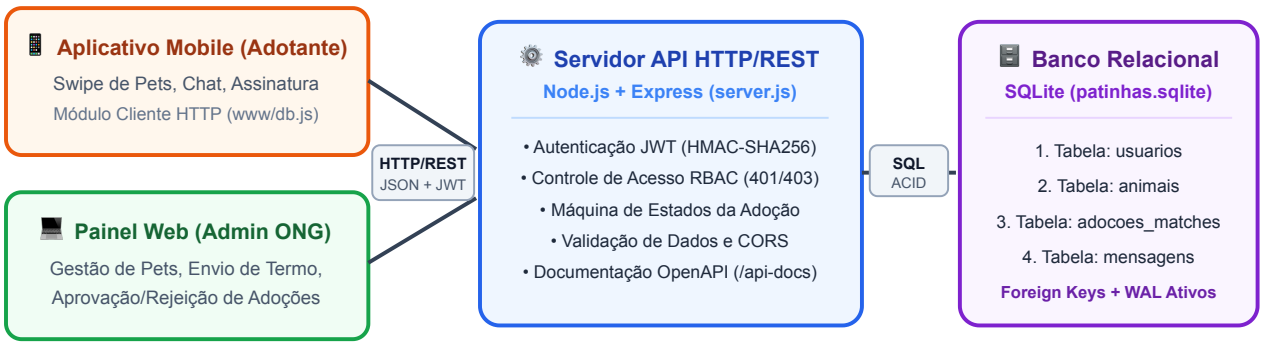


Figura 1 — Arquitetura distribuída do sistema Patinhas: Aplicativo/Painel → API HTTP/REST → Banco de Dados Relacional SQLite.

2. Modelagem e Persistência de Dados Relacional

2.1 Entidades, Relacionamentos e Regras de Integridade

O esquema relacional foi definido no arquivo `backend/schema.sql` e instanciado automaticamente por `backend/database.js` com suporte explícito a chaves estrangeiras (`PRAGMA foreign_keys = ON;`). O modelo contempla quatro entidades principais:

Entidade (Tabela)	Chaves e Atributos Principais	Relacionamentos (FK)	Regras de Integridade
`usuarios`	`id` (PK), `nome`, `cpf`, `telefone`, `email`, `senha_hash`, `perfil`, `avatar_url`	1:N com `animais` 1:N com `adocoes_matches` 1:N com `mensagens`	`UNIQUE(email)`, `UNIQUE(cpf)`, `CHECK(perfil IN ('adotante','ong'))`, senha com hash `scrypt`.
`animais`	`id` (PK), `nome`, `especie`, `raca`, `idade`, `porte`, `sexo`, `foto_url`, `status`, `ong_id` (FK)	N:1 com `usuarios(id)` via `ong_id` (`ON DELETE RESTRICT`)	`CHECK(status IN ('disponivel', 'em_processo', 'adotado'))`, `CHECK(porte IN ('Pequeno','Médio','Grande'))`.
`adocoes_matches`	`id` (PK), `animal_id` (FK), `adotante_id` (FK), `status`, `termo_enviado`, `termo_assinado`, `assinatura_digital`	N:1 com `animais(id)` N:1 com `usuarios(id)` (`ON DELETE CASCADE`)	`UNIQUE(animal_id, adotante_id)` impede matches duplicados; `CHECK` de estados válidos.
`mensagens`	`id` (PK), `match_id` (FK), `remetente_id` (FK), `remetente_perfil`, `texto`, `is_termo`, `horario`	N:1 com `adocoes_matches(id)` N:1 com `usuarios(id)`	`CHECK(remetente_perfil IN ('user','ong'))`, `CHECK(length(trim(texto)) >= 1)`.

2.2 Justificativa da Escolha do Banco de Dados Relacional (SQLite)

Optou-se pelo banco de dados relacional **SQLite 3** integrado nativamente ao motor do Node.js (módulo `node:sqlite` / `DatabaseSync`) pelos seguintes motivos técnicos e arquiteturais:

1. Conformidade Relacional e Transações ACID: O SQLite implementa o padrão SQL relacional completo, garantindo atomicidade, consistência, isolamento e durabilidade (ACID), além de impor restrições de chave estrangeira (`FOREIGN KEY`), unicidade (`UNIQUE`) e validação de domínio (`CHECK`) diretamente no nível do banco.

2. Persistência Real em Disco e Portabilidade Acadêmica: Todos os dados são gravados de forma durável no arquivo binário `backend/patinhas.sqlite` operando em modo *Write-Ahead Logging* (`PRAGMA journal_mode = WAL;`). Ao reiniciar o servidor Node.js, todas as contas criadas, animais cadastrados, matches, assinaturas de termos e mensagens de chat permanecem intactos, sem depender de containers externos complexos para avaliação da banca.

3. Carga Inicial Automatizada (Seeds): Ao iniciar pela primeira vez, `backend/database.js` popula automaticamente 3 usuários (incluindo a conta administrativa `admin@ong.com` e o adotante `lucas@ifrn.edu.br`), 6 animais disponíveis e um processo de adoção com histórico de mensagens para testes imediatos.

3. Desenvolvimento da API HTTP/REST e Máquina de Estados

3.1 Máquina de Estados do Processo de Adoção

O fluxo principal de negócio do sistema Patinhas regula o ciclo de vida de uma adoção responsável em 5 estados bem definidos, validados de forma estrita pelo servidor antes de qualquer transição (retornando código `422 Unprocessable Entity` caso uma transição inválida seja solicitada, como tentar aprovar uma adoção sem que o Termo de Adoção Responsável tenha sido assinado digitalmente):

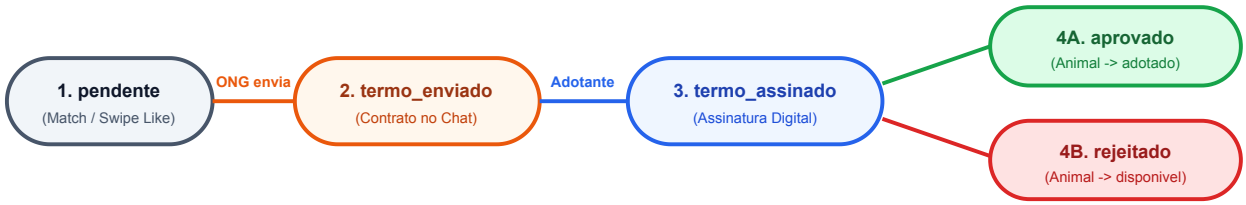


Figura 2 — Máquina de estados finitos do processo de adoção e transições controladas pela API REST.

3.2 Justificativa da Arquitetura REST em Comparação ao SOAP

Para a construção da camada de comunicação distribuída do projeto Patinhas, escolheu-se o estilo arquitetural **REST (Representational State Transfer)** sobre HTTP com payloads em formato **JSON**, em detrimento do protocolo **SOAP (Simple Object Access Protocol)**. A decisão fundamenta-se nos seguintes critérios técnicos:

Critério Técnico	Arquitetura REST (Escolhida no Projeto)	Protocolo SOAP
Formato de Dados e Overhead de Rede	Utiliza **JSON**, formato leve e compacto, reduzindo drasticamente o consumo de banda e bateria em dispositivos móveis (Android/Web).	Exige envelopes **XML** verbosos (`<soap:Envelope>` , `<soap:Header>` , `<soap:Body>`), aumentando o tamanho das mensagens e a latência.
Semântica do Protocolo HTTP	Aproveita nativamente os verbos HTTP (`GET` , `POST` , `PUT` , `PATCH` , `DELETE`) e códigos de status padronizados (`200` , `201` , `400` , `401` , `403` , `404` , `409` , `422`).	Opera geralmente encapsulado apenas em requisições `POST` , ignorando a semântica de cache e status nativos do HTTP (erros vêm em `SOAP Fault`).
Acoplamento e Consumo no Frontend	Nativo no JavaScript moderno via `fetch()` e `JSON.parse()` no módulo `db.js` , sem necessidade de bibliotecas pesadas de parsing no celular.	Exige contratos rígidos em **WSDL** e bibliotecas especializadas de serialização/parsing XML, dificultando a evolução ágil das telas.
Escalabilidade e Estado (Stateless)	Totalmente *stateless*: cada requisição carrega seu próprio token **JWT Bearer**, facilitando escalabilidade horizontal e cache.	Frequentemente acoplado a extensões complexas como *WS-Security* e estado de sessão pesado no servidor.

4. Identificação, Autenticação JWT e Controle de Acesso (RBAC)

O servidor substituiu qualquer login simulado por autenticação real persistida na tabela `usuarios` . As senhas são armazenadas exclusivamente sob derivação criptográfica (`crypto.scryptSync` com salt) e verificadas

com comparação em tempo constante (`crypto.timingSafeEqual`). Ao autenticar em `POST /api/auth/login` , o servidor emite um token **JWT (HMAC-SHA256)** contendo `id` , `email` e `perfil` (`adotante` ou `ong`).

- **Tentativas sem autenticação (`401 Unauthorized`):** O middleware `authenticateToken` intercepta rotas protegidas e recusa imediatamente requisições sem cabeçalho `Authorization: Bearer <token>` ou com assinatura JWT inválida/expirada.
- **Tentativas com permissão insuficiente (`403 Forbidden`):** O middleware `requireRole('ong')` valida no servidor o perfil do usuário autenticado. Um usuário `adotante` não consegue cadastrar/excluir animais (`POST/DELETE /api/pets`), enviar termos (`POST /api/matches/:id/enviar-termo`) nem aprovar adoções (`PATCH /api/matches/:id/status`). Além disso, na rota `GET /api/matches` , um adotante consulta exclusivamente os seus próprios processos (filtrados por `adotante_id = req.user.id`).

5. Resumo das Rotas Documentadas no OpenAPI 3.0 (Swagger)

A especificação completa encontra-se em `backend/openapi.json` e na interface interativa `swagger.html` (ou em `http://localhost:3000/api-docs` quando o servidor está em execução):

Método	Caminho (Endpoint)	Permissão / Auth	Descrição e Códigos HTTP Retornados
POST	`/api/auth/register`	Público	Cadastra novo usuário (adotante ou ONG) e retorna JWT (`201` , `400` , `409`).
POST	`/api/auth/login`	Público	Autentica e-mail/senha no SQLite e emite token JWT (`200` , `400` , `401`).
GET	`/api/auth/me`	JWT (Qualquer)	Retorna dados do usuário logado a partir do token (`200` , `401`).
GET	`/api/pets` e `/api/pets/:id`	Público	Lista catálogo de animais ou detalhes por ID (`200` , `404`).
POST	`/api/pets`	JWT (Perfil: `ong`)	Cadastra novo animal vinculado ao `ong_id` autenticado (`201` , `400` , `401` , `403`).
PUT / DEL	`/api/pets/:id`	JWT (Perfil: `ong`)	Atualiza ou remove um animal do banco de dados (`200` , `401` , `403` , `404`).
GET	`/api/matches`	JWT (Adotante/ONG)	Lista matches do adotante logado ou todos os matches da ONG (`200` , `401`).
POST	`/api/matches`	JWT (Adotante)	Cria match no estado `pendente` e inicia chat com a ONG (`201` , `404` , `409` , `422`).
POST	`/api/matches/:id/enviar-termo`	JWT (Perfil: `ong`)	Transição `pendente` → `termo_enviado` (`200` , `401` , `403` , `404` , `422`).

Método	Caminho (Endpoint)	Permissão / Auth	Descrição e Códigos HTTP Retornados
POST	`/api/matches/:id/assinar-termo`	JWT (Dono do Match)	Transição `termo_enviado` → `termo_assinado` (`200` , `400` , `403` , `404` , `422`).
PATCH	`/api/matches/:id/status`	JWT (Perfil: `ong`)	Transição final para `aprovado` ou `rejeitado` (`200` , `400` , `403` , `404` , `422`).
GET / POST	`/api/matches/:id/messages`	JWT (Participantes)	Consulta ou envia mensagens no chat vinculadas ao usuário (`200` , `201` , `403` , `404`).

Como Executar e Testar o Servidor Localmente

```
# 1. Na pasta raiz do projeto, inicie o servidor da API HTTP/REST + Banco SQLite:
node backend/server.js

# 2. Acesse no navegador:
# - Aplicativo / Painel Web Integrado: http://localhost:3000/01-login.html
# - Documentação Interativa Swagger:   http://localhost:3000/api-docs
# - Especificação OpenAPI Bruta:       http://localhost:3000/api/openapi.json
```